# Lab 40: "Clean this up" means three things: show which one you mean

Topic 4: Prompt Engineering & Structured Output · Task 4.2: Apply few-shot prompting to improve output consistency and quality

Runs on Simulator 2: Code Generation with Claude Code · Run in class

Source: SkillCertPro practice exam Part 1, Q23 (unofficial third-party question)

## Commit to an answer first

Your team's Claude Code skill (custom slash command), invoked as /task , is meant to decide whether to investigate, plan, or modify code. In evals, ambiguous requests such as “clean this up”, “look into this error”, and “can you improve this module” produce inconsistent behavior. Detailed written criteria have not stabilized the boundary. What should you change next?

**A.** Add two to four tagged examples showing similar ambiguous requests, the selected response path, and concise reasoning against plausible alternatives.

**B.** Add a stronger instruction that direct file edits must be avoided whenever the developer's request could imply planning.

**C.** Add a large collection of untagged past transcripts so Claude can infer the team's preferred behavior from volume.

**D.** Add a longer checklist that enumerates every phrase that should trigger investigation, planning, or code modification.

## Simulate the solution

```
# In the browser: open Simulator 2, then Lab 40
cd labs
uv run serve.py

# In Claude Code, from the labs folder
/lab 40

# In a terminal, from simulators/simulator-2-code-generation/
uv run casekit.py --case UC-23
```

## What you should see

The three ambiguous requests classify as investigate, plan and modify.

With three <example> blocks in .claude/commands/task.md, the three ambiguous requests classify as investigate / plan / modify.

Open `simulators/simulator-2-code-generation/evidence/` and find it yourself.

## The answer

**A.** Two to four tagged examples with reasoning against the alternatives.

When a prompt fails at an ambiguous decision boundary, targeted examples usually beat more abstract instructions. The useful pattern shows why one action was chosen over a plausible alternative, such as planning instead of editing. Examples should mirror real requests, be relevant and diverse, and sit in their own tags so Claude can tell reusable guidance apart from the live input.

## Why the others fail

**B.** A blanket “avoid edits” rule over-blocks useful work.

**C.** Untagged transcripts can teach accidental patterns, and they cost context.

**D.** Phrase checklists are brittle, and written criteria have already failed here.

## Check yourself

- 'look into this error' -> investigate
- 'can you improve this module' -> plan
- 'clean this up' with a named file -> modify

## Now change one thing

Set LAB_MODEL=claude-sonnet-5-5, rerun, and compare the result and the cost line in evidence/.

Third-party practice material from SkillCertPro, reconstructed from a video transcript in the Tertiary Markdown Vault. Not a real exam item. Wording may differ from the original.